Prototype: Figma + Desktop app

Idea 1: TaxWise — RAG Tax Learning Assistant

Sep 22, 2026 · @Someone

Problem definition

Most students file their first tax return with no formal instruction, and the official information they need is spread across dozens of long, dense documents.

- **Fragmented sources.** The rules for one situation (a first W-2 job plus a scholarship, say) can span Form 1040 instructions, several IRS publications and a state revenue site.
- **Jargon barrier.** Terms like adjusted gross income, standard deduction and withholding are assumed knowledge, not explained.
- **Generic answers.** Search results and blogs rarely match a student's exact situation, and are often outdated or written to sell tax software.
- **Untrustworthy AI.** General chatbots answer confidently but can invent rules or cite the wrong tax year, with no way to check the source.

Motivation: a tool that answers tax questions in plain language, grounded in and linked to the official source text, would make taxes learnable instead of intimidating. It is also a strong test of vibe coding, because a wrong answer is costly and easy to verify against the source.

Target users and use cases

The primary user is a college student filing a federal return for the first or second time.

User	Typical question
First-time filer with a part-time job	"What does each box on my W-2 mean?"
Student with scholarships or tuition	"Is my scholarship taxable? Can I claim an education credit?"
Student claimed as a dependent	"My parents claim me. Do I still need to file?"
Freelancer or gig worker	"What is self-employment tax and why do I owe it?"
International student	"Which form do I file as a nonresident?"

Proposed solution

TaxWise is a desktop app that teaches personal taxes through a retrieval-augmented generation (RAG) chatbot grounded only in official IRS and state documents, with every answer linked to its source passage.

Core features (mid-semester to final):

1. **Cited Q&A chat.** The user asks a question in plain English; the system retrieves the relevant passages from the knowledge base and answers with inline citations that open the exact source page.
2. **Plain-language mode.** A toggle that rewrites answers at a beginner reading level and defines each tax term on hover.
3. **"Explain my form" walkthrough.** The user uploads or picks a sample W-2 or 1098-T, and the app explains each box and where it goes on Form 1040.
4. **Scenario calculator.** A simple estimator (income, filing status, dependency) that computes tax with deterministic code, not the LLM, so numbers are always correct.
5. **Learning quizzes.** Short generated quizzes on topics the user has asked about, to reinforce concepts.

Guardrails: the assistant says "I don't know" when retrieval finds nothing relevant, labels the tax year of every source, and shows a disclaimer that it is an educational tool, not professional tax advice.

Stretch goals: Colorado state tax coverage, a side-by-side "TaxWise vs. plain chatbot" comparison view, and voice input.

System architecture

The system has two pipelines: an offline ingestion pipeline that builds the knowledge base once, and an online query pipeline that runs on every question.

```
flowchart LR
    subgraph Ingestion[Offline ingestion]
        A[IRS PDFs and<br/>state guides] --> B[Parse text<br/>PdfPig]
        B --> C[Chunk by section<br/>+ metadata]
        C --> D[Embed chunks<br/>Voyage AI]
        D --> E[(Vector DB<br/>Qdrant / pgvector)]
    end
    subgraph Query[Online query]
        F[Blazor UI] --> G[ASP.NET Core API]
        G --> H[Semantic Kernel<br/>retrieve + rerank]
        E --> H
        H --> I[Claude answer<br/>with citations]
        I --> F
        G --> J[Tax calculator<br/>C# class]
    end
```

The LLM never computes tax amounts; the calculator module does, and the LLM only explains its output.

Layer	Choice	Why
IDE	Visual Studio 2022	One solution for UI, API and tests
Frontend	Blazor Web App (.NET)	C# end to end, no JavaScript needed
Backend	ASP.NET Core Web API	Standard .NET web stack
AI orchestration	Microsoft Semantic Kernel	Microsoft's C# library for prompts, RAG and vector search
PDF parsing	PdfPig	Open-source .NET PDF text extraction
Vector store	Qdrant (Docker) or PostgreSQL + pgvector	Both have .NET clients
Embeddings	Voyage AI	Claude has no embedding model; Anthropic recommends Voyage
LLM	Claude via Anthropic's C# SDK	Strong "cite or refuse" instruction-following
Tests	xUnit	Unit tests for calculator and retrieval
Hosting	Azure App Service (Azure for Students credits)	One-click publish from Visual Studio

Knowledge base (v1): Publication 17 (your federal income tax), Publication 970 (education tax benefits), Publication 501 (dependents and filing status), Publication 519 (nonresident aliens), and the Form 1040, W-2 and 1098-T instructions. Each chunk stores its source, section heading, page number and tax year.

Query flow: question → optional query rewrite → vector search (top 8) → rerank to top 4 → prompt with passages and a strict "answer only from sources, cite each claim" instruction → answer with clickable citations.

AI development tool strategy

Claude is our main AI for both coding and the app itself; GitHub Copilot in Visual Studio is the specialist for this project because it is built into Visual Studio and strong on .NET, Semantic Kernel and Azure.

Tool	Role	Used for
Claude Code (terminal beside Visual Studio)	Main coding AI	Architecture, multi-file features, RAG pipeline, prompt design, refactors

Tool	Role	Used for
GitHub Copilot in Visual Studio	Specialist (.NET / Azure)	Inline C# completion, xUnit tests, Semantic Kernel setup, Azure deployment
Claude API	In-app AI	Answering questions from retrieved IRS passages

Head-to-head comparisons: for 3 to 5 tasks (the PDF chunker, the tax calculator, the citation UI) we give the same prompt to Claude Code and Copilot and log which result needed fewer fixes. This gives the report hard evidence on where each tool is stronger.

Part of the system	AI tool	Human role
Blazor UI, chat components, styling	Claude Code	Review UX, accessibility
ASP.NET Core endpoints, PDF ingestion	Claude Code, Copilot for completion	Check error handling, API key safety
Semantic Kernel RAG setup	Copilot + Claude Code	Design chunking and prompt rules, run evals
Tax calculator	Both draft, human verifies	Hand-check every rule against the IRS source
Tests and evaluation set	Copilot generates tests	Write the gold answers from the publications

Prompt log: every session is recorded in `/docs/prompt-log.md` with date, tool, prompt, what was generated, whether it worked, and what we changed. We will tag entries as *success*, *partial* or *failure* so we can count them for the report.

What we expect to learn: AI should excel at UI and boilerplate, and struggle with subtle tax rules, chunking strategy and security details such as exposed API keys.

Mid-semester prototype and demo

By the week of Oct 5 we will show a working cited Q&A chat over two IRS publications (Pub 17 and Pub 970).

- ☐ Ingestion script that turns the two PDFs into a Qdrant vector database
- ☐ Minimal chat page that sends a question and shows the answer with citations
- ☐ Clicking a citation shows the source passage and page number
- ☐ Refusal when the question is out of scope (for example, crypto staking)
- ☐ 3 to 5 prompt-log entries to show on a slide, including one AI failure

Demo script (about 2 minutes): ask "Is my scholarship taxable?", open the citation, then ask the same question to a plain chatbot and compare.

Risks, limitations and evaluation

The biggest risk is a confident but wrong tax answer, so evaluation centers on grounding and citation accuracy.

Risk	Mitigation
Hallucinated rules or numbers	Answer only from retrieved text; deterministic calculator for math
Outdated tax year	Tag every chunk with its year; show it in the answer
Poor retrieval from tables in PDFs	Test table-aware parsing; hand-fix key tables
Users treating it as tax advice	Clear disclaimer; link to IRS Free File and VITA clinics
Leaked API keys in AI-written code	Environment variables, secret scanning on GitHub

Evaluation plan: a gold set of about 50 student tax questions with answers written from the publications. We will measure retrieval hit rate (right passage in top 4), answer correctness, citation accuracy, and refusal rate on out-of-scope questions, and compare TaxWise against the same LLM with no retrieval.

Timeline and team split

Dates	Milestone
Sep 22 – Oct 4	Ingestion + basic cited chat (mid-semester prototype)
Week of Oct 5	Mid-semester presentation
Oct 6 – Oct 31	Full knowledge base, plain-language mode, form walkthrough
Nov 1 – Nov 20	Calculator, quizzes, evaluation set and results
Nov 21 – Nov 29	Polish, deploy, rehearse demo
Week of Nov 30	Final presentation and live demo
Dec 10	Report (6+ pages) and GitHub README due

Member	Owns
Member A	Frontend, form walkthrough, deployment
Member B	Ingestion, RAG chain, evaluation

Member	Owns
Both	Tax calculator review, prompt log, report and slides